

Java's Framework Collection

Classe ArrayList

Bib: Java: An Introduction to Problem Solving & Programming, Walter Savitch

Cap. 12: Dynamic Data Structures and Generics

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

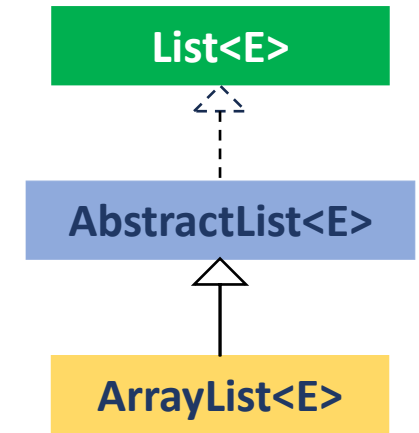
Gabinete F.0.23 do edifício F, LEETC, ADEETC

Slides baseados nos slides da professora Manuela Sousa



ArrayList – o que é?

- A classe `ArrayList` implementa a interface `List`
- Instâncias de `ArrayList` servem os mesmos propósitos que *arrays* com a diferença que podem aumentar ou reduzir a capacidade
 - A vantagem de poder expandir ou reduzir a dimensão tem um custo computacional associado
- Casos de utilização
 - Representação de listas de elementos ordenados de acordo com a posição em que o programador os colocou
 - Acesso não sequencial (acesso *random*)
 - Acesso por índice tal como nos *arrays*: 0 a size-1



ArrayList – instanciação

- Criação com uma capacidade inicial por omissão
`ArrayList<Base_type> v1 = new ArrayList<>();`
- Criação com uma determinada capacidade inicial
`ArrayList<Base_type> v2 = new ArrayList<>(capacity);`
- Ex (cria ArrayList com capacidade para colecionar 100 inteiros):
`ArrayList<Integer> al = new ArrayList<>(100);`
 - Como não é permitido criar coleções de tipos primitivos, neste exemplo usou-se o *wrapper* para o tipo `int`, o tipo `Integer`
- Classe `Arrays` que retorna uma lista a partir de uma sequência de elementos
`List<String> list = Arrays.asList("Olá", "isto", "é", "um", "teste");`
- Deve usar-se uma capacidade inicial com o número total de elementos sempre que se conheça essa informação
 - A expansão tem custo computacional: alocação de *array* com nova capacidade e cópia dos elementos para novo *array*

ArrayList – acrescentar, obter e substituir elementos

- Permite a inserção de `null` e de valores repetidos
- Adicionar um elemento no fim da lista
`boolean add(E elem);` ex: `al.add(10);`
- Adicionar um elemento no meio da lista
`boolean add(int index, E elem);` ex: `al.add(1, 20);`
 - A adição de um elemento no meio da lista empurra os elementos seguintes para a frente
 - Produz a exceção `IndexOutOfBoundsException` se `idx < 0 || idx > size`
- Obter o primeiro e o último elemento da coleção, e o elemento no índice
`E getFirst();    E getLast();    E get(int idx);`
ex: `int value = al.get(1);`
- Substituir o elemento no índice retornando o elemento substituído
`E set(int idx, E elem);` ex: `al.set(1, 30);`
 - Gera exceção `IndexOutOfBoundsException` se `idx < 0 || idx >= size`

ArrayList – remover elementos

- Remover o elemento no índice
`E remove(int idx);` ex: `al.remove(1);`
- Remover elemento se existir
`boolean remove(Object obj);`
ex: `al.remove(Integer.valueOf(20));`
- Remover o primeiro e o último elemento
`E removeFirst();` `E removeLast();`
- Remover todos os elementos da coleção
`boolean removeAll(Collection<?> c);`
- Remover todos os elementos dado o predicado
`boolean removeIf(Predicate<? super E> filter);`
- Remover todos os elementos da lista
`void clear();`

ArrayList – diferença entre *size* e *capacity*

- Os valores de *size* e *capacity* não têm o mesmo significado
 - O valor *size* representa o número de elementos na lista
int size();
 - O valor *capacity* representa o número total de entradas na lista
- A capacidade da lista é aumentada à medida das necessidades
 - Adição de um novo elemento ou elementos e não existir capacidade
- Aumentar a capacidade, se necessário, de forma a suportar um conjunto de elementos especificado pela capacidade mínima
void ensureCapacity(int minCapacity);
- Verificar se a lista está vazia
boolean isEmpty();
- Ajustar a capacidade para o número de elementos presentes na lista (*capacity = size*)
void trimToSize();

ArrayList – pesquisar por um elemento

- Devolver índice da primeira ocorrência do objeto ou -1 se não existir
`int indexOf(Object object);`
- Devolver índice da última ocorrência do objeto ou -1 se não existir
`int lastIndexOf(Object object);`
- Devolver se um objeto existe na lista
`boolean contains(Object object);`

- Os parâmetros serem do tipo `Object` permite que sejam usados objetos que não do tipo parametrizado usado na criação da lista

Exemplo com ArrayList

```
public static void main(String[] args) {  
    // Criação  
    List<Integer> al = new ArrayList<>(100);  
    // Adição  
    System.out.println("isEmpty: " + al.isEmpty());  
    al.add(10); al.add(1,20); al.add(30);  
    System.out.println("size1: " + al.size());  
    al.set(1, 30); System.out.println("size2: " + al.size());  
    // Remoção  
    al.removeFirst(); al.remove(Integer.valueOf(30));  
    System.out.println("size3: " + al.size());  
    // Pesquisa  
    al.add(30);  
    System.out.println("indexOf 30: " + al.indexOf(30));  
    System.out.println("lastIndexOf 30: " + al.lastIndexOf(30));  
    System.out.println("contains 10: " + al.contains(10));  
}
```

```
isEmpty: true  
size1: 3  
size2: 3  
size3: 1  
indexOf 30: 0  
lastIndexOf 30: 1  
contains 10:  
false
```

ArrayList – método clone

- O método `clone` é usado para criar uma cópia de um `ArrayList`

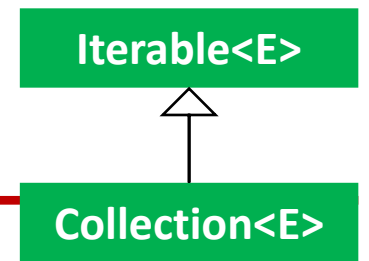
```
ArrayList<Integer> cpAl = (ArrayList<Integer>)al.clone();
```

- Cria uma nova instância de `ArrayList` e copia o estado do objeto atual para o objeto criado

```
System.out.println("cpAl.equals(al): "+cpAl.equals(al)); // produz true  
System.out.println("cpAl == al: " + (cpAl == al));      // produz false
```

- Os elementos contidos no `ArrayList` são compartilhados pelas duas listas (equivalente ao que acontece quando se clona um objeto com referências para outros objetos – são copiadas as referências mas não são clonados os objetos referenciados)
 - Esta cópia chama-se *cópia superficial* (*shallow copy*)
-
- Uma *cópia profunda* (*deep copy*), para além de clonar o objeto atual, clona igualmente os objetos referenciados

Interfaces Iterable, Iterator e Consumer



- Qualquer coleção que implemente a interface Iterable é iterável permitindo que a coleção seja usada em instruções *for-each*
- Um iterador permite percorrer todos os elementos de uma coleção
- Deve produzir a exceção UnsupportedOperationException qualquer operação opcional não implementada

Iterable<T>
+iterator(): Iterator<T> +forEach(action: Consumer<? super T>) {default} +spliterator(): Spliterator<T> {default}

Iterator<T>
+hasNext(): boolean +next(): T +remove() {default, optional} +forEachRemaining(action: Consumer<? super T>)

Consumer<T>
+accept(t: T) +andThen(after: Consumer<? super T>): Consumer<T> {default}

Exemplo com várias opções para iterar sobre ArrayList

```
static void printWithGet(List<String> list)
```

```
{ for (int i = 0; i < list.size(); i++) System.out.println(list.get(i)); }
```

Inadequado para
um LinkedList

```
static void printWithIterator(List<String> list)
```

```
{ Iterator<String> it = list.iterator(); while (it.hasNext()) System.out.println(it.next()); }
```

```
static void printWithForEachInstruction(List<String> list)
```

```
{ for (String s : list) System.out.println(s); }
```

Internamente usa
um iterador

```
static void printWithIterableForEachWithLambda(List<String> list)
```

```
{ list.forEach(v -> System.out.println(v)); }
```

```
static void printWithForEachWithMethodReference(List<String> list)
```

```
{ list.forEach(System.out::println); }
```